1. ATIP as an algorithm: design note

This note turns a published procedure for classifying arguments into a program. It labels arguments with their type, first against human labels in the note-lab annotation project, later in-game against the player's own arguments.

1.1. Background: the PTA and ATIP

The *Periodic Table of Arguments* (PTA, version 3.1) classifies arguments into 50 types (Wagemans; see Section 1.11). In the PTA an argument consists of three statements:

- a **conclusion**, the claim being argued for;
- a **premise**, the reason given for it;
- a **lever**, the usually unstated statement that says *how* the premise makes the conclusion acceptable ("rising prices are a CAUSE of strikes").

Each statement has a **subject**, what it is about, and a **predicate**, what it says of the subject. An argument's type is the unique combination of three parameters:

- **form**: which elements the conclusion and premise share. There are four forms, written α, β, γ, δ (Section 1.5.3).
- **substance**: what kind of statements or terms are involved (Section 1.5.4).
- **lever**: the relation the lever statement asserts, named by a keyword in capitals such as CAUSE or ANALOGOUS (Section 1.5.5).

The *Argument Type Identification Procedure* (ATIP, version 6) determines these in six steps (Wagemans 2026): (1) label the textual elements, (2) reformulate the argument as "conclusion because premise", (3) determine the form, (4) the substance, (5) the lever, and (6) name the type. This note follows the same six steps.

Each type has a name and a symbol: "argument from cause", `Cau`. We write a classified argument as a **code** of form, substance and symbol: `α FF Cau` is an alpha argument whose conclusion and premise are both statements of fact, of the type argument from cause. Section 1.12 lists all 50 types.

1.2. Scope

In scope:

- Typing one argument, a single premise ⟶ conclusion pair, with a code, plus a confidence and the runner-up codes. These three fields are the **output record**.
- Measuring each part of the procedure against human labels.
- The shape of the output that the game consumes.

Out of scope:

- Judging whether an argument is *good*. That is a later step that reads this one's output.
- How the game uses the output: deciding which of its keys an argument unlocks (sketched in Section 1.8 only as an interface).
- Explication of missing premises in free text (see Section 1.9). In the lab, reconstructed premises come from the human labels.

1.3. Principle: a deductive program with statistical oracles

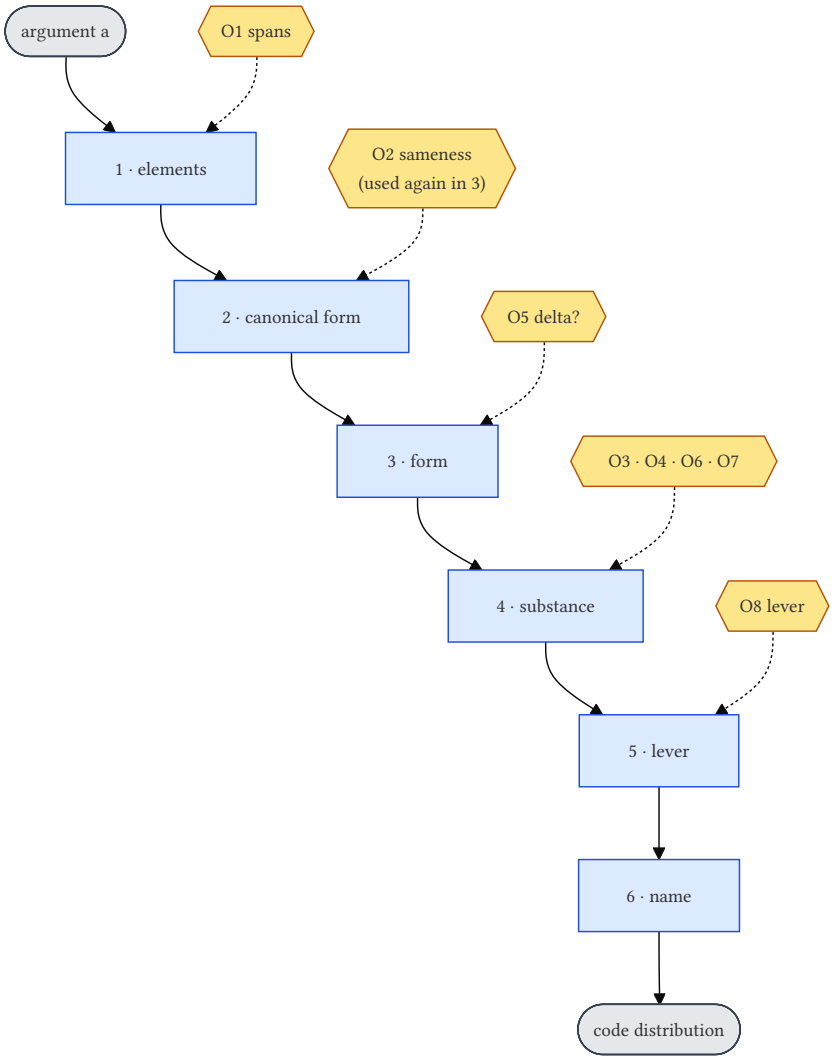
ATIP is mostly a decision table. Steps 3 and 6 are lookups; steps 4 and 5 pick from small closed sets. So the program is deterministic, and every judgement it can't make by rule is an **oracle**: a named function from an argument a and its surrounding text ctx (both defined in Section 1.4) to a

probability distribution over a small closed set:[1]

$$O_i : (a, \mathrm{ctx}) \to \Delta(V_i), \quad |V_i| \leq 10$$

A rule is an oracle that returns a point mass.[2] Rules and models are therefore interchangeable per oracle, so each oracle has its own arms[3] in the lab (rule baseline, embedding model, NLI model, …). Every statistical judgement is localised, named and testable on its own.

There are eight oracles, O_1 to O_8. Each is introduced at the step that uses it, and Section 1.6 summarises them. The whole pipeline, with deterministic steps as **blue rectangles** and oracles as **yellow hexagons**:



1.4. Input

An argument is $a = (c, \rho, \mathrm{ctx})$: the conclusion c, the premise ρ (rho, for "reason") and the surrounding text ctx. c and ρ are spans of text.

- **Lab:** arguments come from INCEpTION, the annotation tool. Labellers mark argumentative spans and draw relations between them on an `Argument`

[1] V_i is the oracle's set of possible answers; $\Delta(V_i)$ is the set of probability distributions over it: one probability per answer, summing to 1.

[2] A distribution with all its probability on one answer: 1 for its answer, 0 for the rest.

[3] An arm is one way of answering an oracle, e.g. a rule baseline or a particular model with a particular configuration.

`relation` layer. An `inference` relation runs from the premise span (the relation's *governor*) to the conclusion span (its *dependent*); each such edge is one argument. A premise the labeller reconstructed rather than found in the text is a zero-width span flagged `reconstructed`, with its wording in the span's `note`. Premises that support a conclusion only jointly share a `group` id; we type each of their edges separately and flag it as linked.

- **Game:** c and ρ come from an extra oracle over the player's line and the dialogue so far (Section 1.8).

1.5. Pipeline

Throughout, subscript c marks the conclusion and ρ the premise, and the conclusion always comes first.

Running example (made up, like all examples here):

"I'd say the minister did a great job, since she raised the university budget."

Probabilities in the examples are illustrative, not measured.

1.5.1. Step 1: elements

Split each statement into subject and predicate, and mark the surrounding elements. **O1** finds these spans.

$$E(c) = (s_c, p_c), \quad E(\rho) = (s_\rho, p_\rho), \quad M \subset \text{ctx}$$

s is a subject and p a predicate. Write Su for the set of subject terms and Pr for the set of predicate terms, so $s_c, s_\rho \in \text{Su}$ and $p_c, p_\rho \in \text{Pr}$. M is the set of **surrounding elements**, which ATIP sorts into four kinds of marker:

- **coherence**: marks which statement is premise and which conclusion ("because", "since", "so");
- **structure**: relates the argument to others ("in addition", "secondly");
- **illocutive**: the arguer's stance towards the claim ("I'd say", "I don't believe");
- **perlocutive**: the effect the arguer intends ("you should accept that").

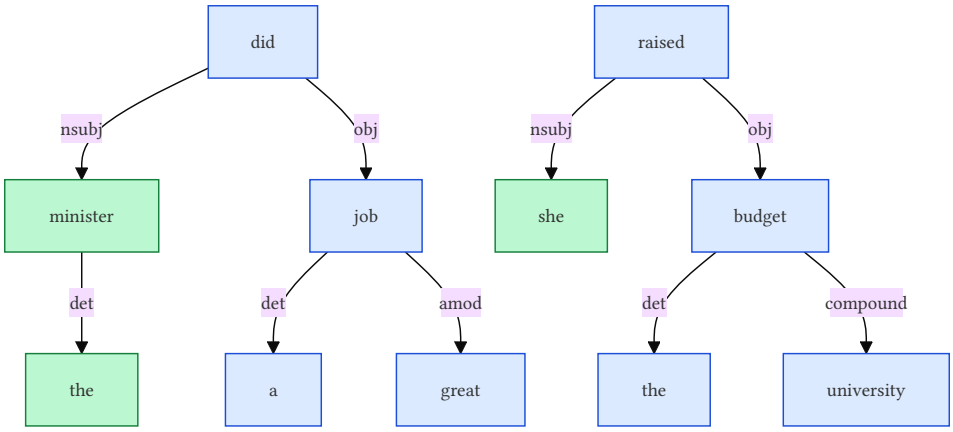
INCEpTION has no subject/predicate spans, so this is automated even in the lab. Rule baseline: dependency parse,[4] subject = the `nsubj`/`nsubj:pass` subtree, predicate = the rest of the clause; copulas[5] kept with the predicate. Markers come from a lexicon of the four kinds.

Step 1 also records q_ρ: the whole proposition the premise speaks of, if any ("Eurostat said *that the economy grew*"). Step 3 uses it to detect delta arguments. Rule baseline: the clausal complement of a reporting verb ("Eurostat said *that the economy grew*") or an anaphoric "so"/"that"/"this" ("Eurostat said *so*"), resolved to the clause it stands for. Otherwise q_ρ is empty.

Example. The human-labelled ("gold") spans give the conclusion "the minister did a great job" and the premise "she raised the university budget". Their parses, with **subject** and **predicate** words coloured:

[4] A parse that links each word to the word it grammatically depends on and labels the link: `nsubj` is a nominal subject, `nsubj:pass` the subject of a passive verb.

[5] Linking verbs such as "is", "are", "seems".




```
conclusion: {s_c: "the minister", p_c: "did a great job"}
premise:    {s_ρ: "she",          p_ρ: "raised the university budget"}
markers:
  - {text: "I'd say", kind: illocutive}
  - {text: "since",   kind: coherence}
```

1.5.2. Step 2: canonical form

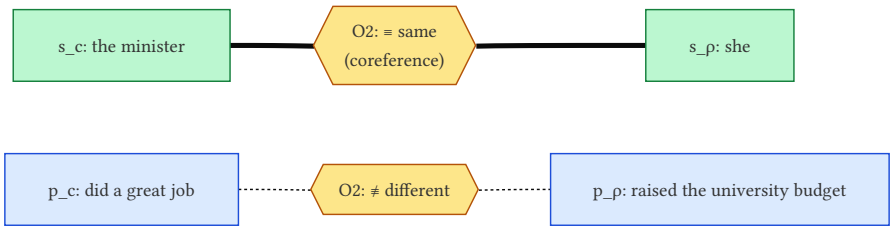
ATIP's step 2 rewrites the argument into the canonical sentence "conclusion because premise". Its transformations include moving the conclusion first, replacing the coherence marker by "because", dropping the other markers, resolving anaphora (replacing "she" by "the minister"), making implicit content explicit and switching between active and passive voice. Their purpose is to make shared subjects and predicates *literally* shared, because step 3 decides the form from what is shared.

Here the canonical form $\kappa(a)$ is not a rewritten string. It is the tuple from step 1 plus an equivalence relation $\equiv$ on terms:

$$\kappa(a) = \big((s_c, p_c), (s_\rho, p_\rho), \equiv\big)$$

$t_1 \equiv t_2$ means "t_1 and t_2 are the same subject or predicate after anaphora resolution, passivization and paraphrase"; $\not\equiv$ is its negation. Oracle **O2** judges it. Making $\equiv$ explicit means it doesn't matter whether two terms happen to share their wording. The canonical sentence in the output record is flattened from the tuple, using one representative wording per equivalence class.

Example. O2 compares the subjects and the predicates pairwise:



Flattening, with the transformations recorded as ATIP asks:

```yaml
canonical: >-
  the minister did a great job
  because the minister raised the university budget
transformations:
  - omission of illocutive marker "I'd say"
  - substitution of "since" by "because"
  - 'anaphora resolution: "she" → "the minister"'
```

Delta reading. Some arguments rest on who said something or what would follow from it, not on its content: "the economy has grown, because Eurostat said so". For these the PTA splits both statements differently: the subject is the whole proposition. When q_ρ is non-empty, κ also produces this second split:

$$\kappa^{\delta}(a) = \big((c, \zeta), (q_\rho, Z)\big), \quad \zeta \in \mathbb{A} = \{A, U\}$$

In the conclusion, c as a whole is the subject and its predicate ζ is A (*acceptable*) or U (*unacceptable*); $\mathbb{A}$ is the set of the two. In the premise, q_ρ is the subject and Z is what the premise says of it ("was said by Eurostat"). ζ is usually implicit, so flattening writes it in brackets, as ATIP does: "the economy has grown [is acceptable (A)] because that was said by Eurostat". Step 3 decides which split is used.

Passivization works on the parse. In "the budget was raised by the minister", the subject is `nsubj:pass` = "the budget"; when there is an agent (`obl:agent` = "the minister"), the rule baseline also tries the active reading (subject "the minister"), so a shared subject isn't missed.

1.5.3. Step 3: form

ATIP distinguishes four forms by what the conclusion and premise share. It writes them with schematic letters: a and b are subjects, X and Y predicates (X the conclusion's, Y the premise's), q a whole proposition and Z the premise's predicate in a delta argument. In our terms $a = s_c$, $b = s_\rho$, $X = p_c$, $Y = p_\rho$.

form	ATIP schema	shared	test here
α alpha	a is X because a is Y	subject	$s_c \equiv s_\rho, p_c \not\equiv p_\rho$
β beta	a is X because b is X	predicate	$p_c \equiv p_\rho, s_c \not\equiv s_\rho$
γ gamma	a is X because b is Y	nothing	$s_c \not\equiv s_\rho, p_c \not\equiv p_\rho$
δ delta	q is A/U because q is Z	the whole proposition	$c \equiv q_\rho$ (below)

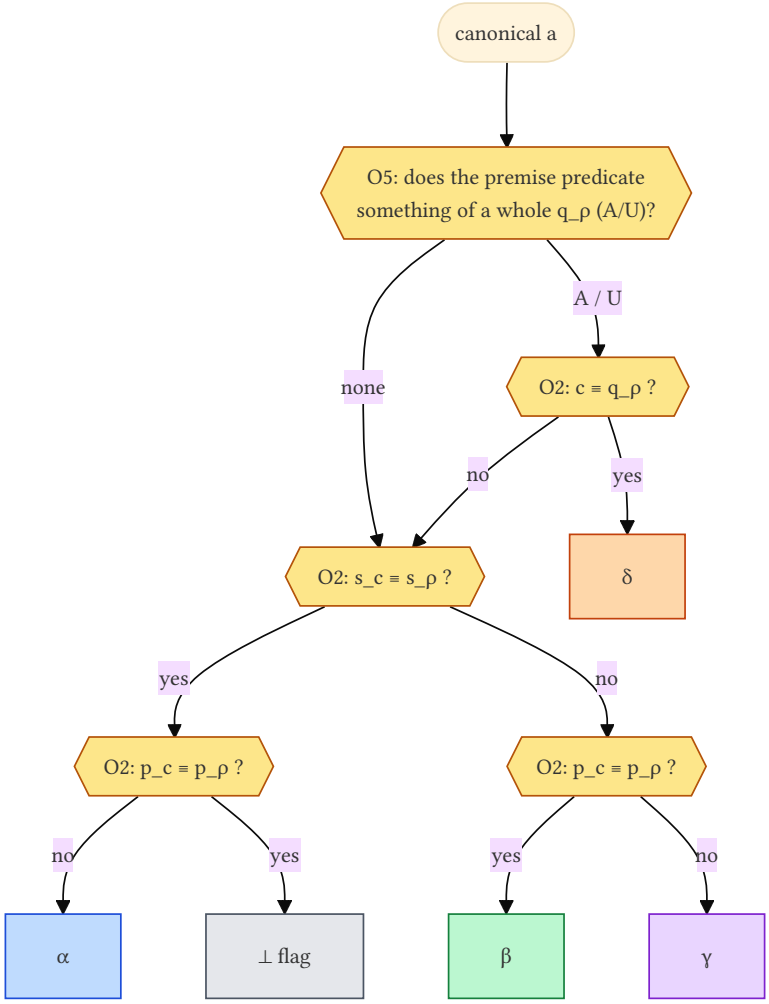
Delta covers the source-based arguments: appeals to an expert, to popularity, threats, personal attacks. Delta is structural: it is the alpha pattern, a shared subject, where the shared subject is a whole proposition and the conclusion's predicate is acceptability. Under the delta reading of step 2:

$$\delta \quad \text{iff} \quad q_\rho \neq \varnothing \wedge c \equiv q_\rho \wedge \zeta \in \mathbb{A}$$

($\varnothing$: step 1 found no such proposition.) Two judgements decide it: **O5**, whether the premise really predicates something of q_ρ as a whole, and with which ζ (values A, U, none); and **O2**, whether q_ρ is the conclusion. Delta is checked first because its reading re-splits both statements; if it holds, the ordinary split isn't used. The form function f maps an argument to its form:

$$f(a) = \begin{cases} \delta & \text{if } O_5(a) \in \mathbb{A} \wedge c \equiv q_\rho \\ \alpha & \text{if } s_c \equiv s_\rho \wedge p_c \not\equiv p_\rho \\ \beta & \text{if } p_c \equiv p_\rho \wedge s_c \not\equiv s_\rho \\ \gamma & \text{if } s_c \not\equiv s_\rho \wedge p_c \not\equiv p_\rho \\ \bot & \text{if } s_c \equiv s_\rho \wedge p_c \equiv p_\rho \end{cases}$$

$\bot$ (both shared) is a rephrase or begging the question; ATIP has no form for it, so we flag it and stop. As a decision tree, with oracles as **hexagons** and each form in its own colour:



The oracles return probabilities, so $\equiv$ holds with some probability rather than outright. Treating the sameness judgements as independent, each leaf's probability is the product along its path. In the sketch, `O2(t1, t2)` is $P(t_1 \equiv t_2)$:

```
def form(a):
    zeta = O5(a)              # distribution over {"A", "U", "none"}
    d  = (1 - zeta["none"]) * (O2(a.c, a.q_ρ) if a.q_ρ else 0)  # P(δ)
    ss = O2(a.s_c, a.s_ρ)     # P(subjects are the same)
    pp = O2(a.p_c, a.p_ρ)     # P(predicates are the same)
    return {
        "δ": d,
        "α": (1 - d) * ss       * (1 - pp),
        "β": (1 - d) * (1 - ss) * pp,
        "γ": (1 - d) * (1 - ss) * (1 - pp),
        "⊥": (1 - d) * ss       * pp,
    }
```

Examples, one per outcome:

argument	shared	form
the minister did a great job because *she* raised the university budget	subject (she ≡ the minister)	α
cycling on the lawn is prohibited because all activities on the lawn are prohibited	predicate (is prohibited)	β
peace can be started by the truth because war can be started by lies	nothing	γ
the economy has grown, because Eurostat said so	the proposition: q_ρ = "so" = "the economy has grown" $\equiv c$; $\zeta = A$	δ
the minister did well because the minister did a good job	both	⊥

Running example: $P(\delta) = 0.03$, $P(s \equiv) = 0.95$, $P(p \equiv) = 0.04$, so $P(\alpha) \approx 0.88$, $P(\bot) \approx 0.04$, $P(\gamma) \approx 0.05$.

1.5.4. Step 4: substance

The values the substance can take depend on the form:

- **α**: the type of each predicate. **F** (fact): a claim checkable by observation or evidence, whether or not it is true. **V** (value): a judgement against some standard (aesthetic, moral, legal, ...). **P** (policy): a recommendation to act, often with "should". Oracle **O3**.
- **β**: the quantity of each subject. **s** (singular): one individual. **p** (particular): some members of a group. **u** (universal): all of them. Oracle **O4**. (This p is unrelated to the predicates p_c, p_ρ.)
- **γ**: **i** (identical) if the subjects relate to each other as the predicates do, otherwise **d** (different). Oracle **O7**.
- **δ**: the perspective of the premise's source. **I**: first person, the arguer. **II**: second person, the addressee (threats, rewards, their commitments). **III**: third person (an expert, a witness, the public). Oracle **O6**.

For α and β the substance is a pair, written conclusion first: **VF** is a value conclusion from a factual premise. The substance function S:

$$S(a) = \begin{cases} (O_3(p_c), O_3(p_\rho)) \in \{F, V, P\}^2 & \text{if } \alpha \\ (O_4(s_c), O_4(s_\rho)) \in \{s, p, u\}^2 & \text{if } \beta \\ O_7(R_s, R_p) \in \{i, d\} & \text{if } \gamma \\ O_6(Z) \in \{\mathrm{I}, \mathrm{II}, \mathrm{III}\} & \text{if } \delta \end{cases}$$

For gamma, R_s is the relation between the subjects (s_ρ to s_c) and R_p the relation between the predicates (p_ρ to p_c); the substance is i iff $R_s = R_p$. O7 judges that comparison, so its input is the pair of relations, not the whole argument. For delta, the perspective is read off the premise's predicate Z (who says, threatens, commits, ...).

Rule baselines:

```
def O3(p):  # predicate type: F / V / P
    if p.modal in {"should", "ought", "must"} or p.is_imperative:
        return "P"
    if p.head in EVALUATIVE or any(m in EVALUATIVE for m in p.modifiers):
        return "V"
    return "F"

def O4(s):  # subject quantity: s / p / u
    if s.det in {"all", "every", "each"} or s.is_generic_plural:
        return "u"
    if s.det in {"some", "a few", "many", "most"}:
        return "p"
    return "s"  # proper noun, "the X", pronoun, activity

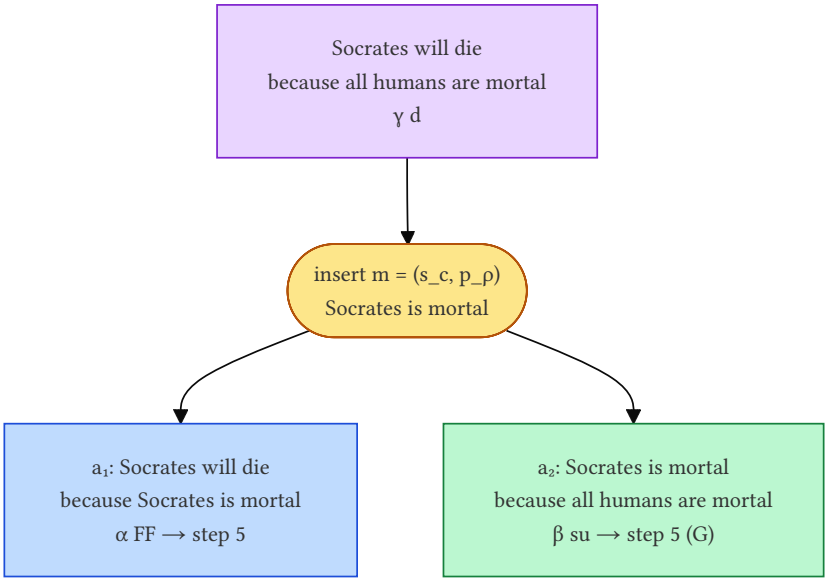
def O6(Z):  # perspective of the source in Z: I / II / III
    match Z.source.person:
        case 1: return "I"
        case 2: return "II"
        case _: return "III"
```

Examples:

form	input	oracle says	substance
α	p_c “did a great job” p_ρ “raised the university budget”	V (“great” is evaluative) F	**VF**
β	s_c “cycling on the lawn” s_ρ “all activities on the lawn”	s u (“all”)	**su**
γ	R_s: war ⟶ peace R_p: lies ⟶ truth	both are OPPOSITE, so $R_s = R_p$	**i**
δ	Z “was said by Eurostat”	third person	**III**

Condensed gamma. A gamma argument with substance d is usually an argument with a missing middle step. ATIP says to try inserting it before concluding that the premise is irrelevant, which is the type *non sequitur* (Non). Build the middle statement $m = (s_c, p_\rho)$ (“a is Y”), then type $a_1 = (c, m)$ as α and $a_2 = (m, \rho)$ as β. Depth one; no further recursion. The open problem is scoring whether the insertion makes the premise relevant (Section 1.9). The result is two output records, linked.

Example: “Socrates will die because all humans are mortal”. Nothing is shared (Socrates ≠ all humans, will die ≠ are mortal), so γ, and the two relations aren’t the same, so O_7 says d. Insert m = “Socrates is mortal”:



1.5.5. Step 5: lever

The lever is the argument’s third statement (Section 1.1), so step 5 chooses a statement, not just a keyword. Write F for the form and t for the substance. $K(F,t)$ is the set of keywords the PTA lists for that form and substance (Section 1.12); for example $K(\alpha, \mathrm{VF}) = \{\mathrm{CRITERION}, \mathrm{DEFINITION}, \mathrm{SUBSUMED}\}$. Each keyword k names a relation, and the candidate levers are its instances ℓ_k on the argument’s own terms:

$$L(F,t) = \{\ell_k : k \in K(F,t)\}$$

The terms a lever relates are fixed by the form:

form	ℓ_k	relates
α	$k(Y,X)$	premise predicate, conclusion predicate: (Pr, Pr)
β	$k(a,b)$ or $k(b,a)$	the subjects: (Su, Su); the keyword fixes the order (a is ANALOGOUS to b; b is an EXAMPLE of a)
γ	$k(b,a) \wedge k(Y,X)$	one relation holding on both pairs
δ	$k(Z,\zeta)$	premise predicate, acceptability: (Pr, 𝔸)

The candidates are deductive. $K(F,t)$ is empty for α FV/VP/FP. The PTA is open to new types, so a lever outside the table is reported as `novel` rather than forced into a listed type.

1. **Cue override.** If the context states a keyword (“is a SIGN of”, “is ANALOGOUS to”) between the argument’s own terms, return that keyword with probability 1.
2. **Otherwise oracle O8.** Write out each candidate lever statement ℓ_k as a sentence, the hypothesis h_k (“rising prices are a CAUSE of the strike”). Score each with an NLI model[6] as entailment of h_k

[6] Natural language inference: given a text and a hypothesis, the model scores whether the text entails the hypothesis (makes it follow), contradicts it, or neither.

[7] softmax turns a list of scores into probabilities that sum to 1, keeping their order: $\mathrm{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$.

by the context. Normalise over $K(F,t) \cup \{\text{none}\}$ with a softmax;[7] `none` above a threshold gives `novel`.

$$l(a) = \text{softmax}_{k \in K(F,t) \cup \{\text{none}\}} \text{entail}(\text{ctx}, h_k)$$

```
def lever(a, form, substance):
    K = LEVERS[form][substance]              # table lookup
    if not K:
        return {"novel": 1.0}
    for k in K:
        if cue(k).links(a.terms, a.ctx):     # e.g. "is a SIGN of"
            return {k: 1.0}
    h = {k: fill(TEMPLATE[k], a) for k in K}
    h["none"] = "none of these"
    return softmax({k: entail(a.ctx, h_k) for k, h_k in h.items()})
```

Running example, α VF. The candidates and their filled-in hypotheses (Y = "raising the university budget", X = "doing a great job"):

k	h_k	P
CRITERION	raising the university budget is a criterion for doing a great job	0.62
SUBSUMED	raising the university budget can be subsumed under doing a great job	0.21
DEFINITION	raising the university budget is the definition of doing a great job	0.09
none		0.08

With a cue instead: "raising the budget is the very *definition* of a great job" skips the model and returns DEFINITION with probability 1.

1.5.6. Step 6: name

$$n : (F, t, k) \to N \cup \{\text{novel}\}$$

N is the set of the PTA's 50 type names. n is a pure lookup in Section 1.12, which reproduces ATIP's Table 7. `γ d` has no keyword (k is empty) and maps to `Non`, which is a type, not a missing value. Within a form the keyword alone fixes the name, as the PTA names types after the lever's KEYWORD; t is kept as a consistency check (a keyword outside $K(F,t)$ is an error upstream).

```
NAMES[("α", "VF", "CRITERION")]  # "argument from criterion (Cr)" -> "α VF Cr"
NAMES[("β", "su", "GENUS")]      # "argument from genus (G)"      -> "β su G"
NAMES[("γ", "d",  None)]         # "non sequitur (Non)"           -> "γ d Non"
NAMES.get(("α", "FV", None))     # None                           -> "novel"
```

1.5.7. Composition

The tree is small: at most 4 forms × 9 substances × 10 levers. Enumerate every path and multiply its oracle probabilities to get an exact distribution over codes:

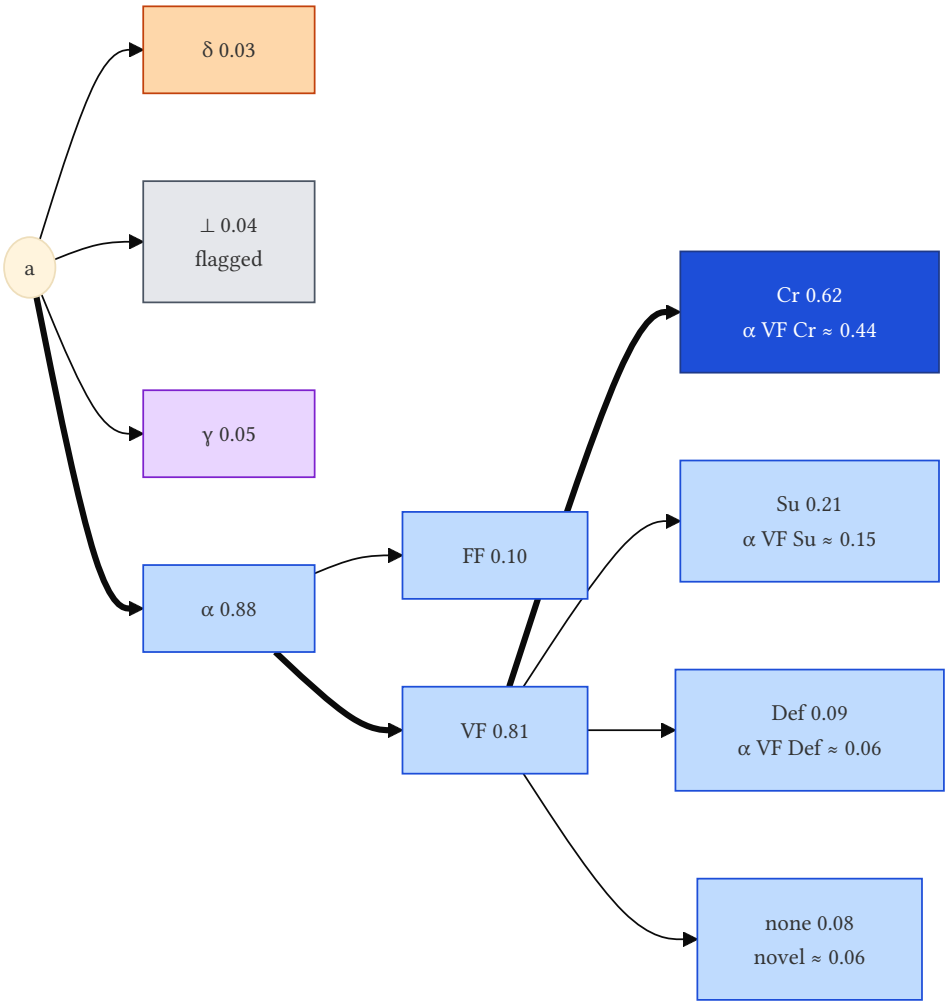
$$P(\text{code}) = \sum_{\text{paths} \to \text{code}} \prod_i P(O_i = v_i)$$

Here v_i is the answer oracle O_i gives on that path. The output record's `code` is the most likely code, `alternatives` the runners-up, and `confidence` the top probability. The product assumes the oracles

are independent; the lab checks whether the resulting confidences are calibrated.[8] Paths whose probability falls below a threshold are pruned (not explored further).

```
def classify(a):
    dist = Counter()
    for F, pF in form(a).items():
        if F == "⊥":
            dist["⊥"] += pF
            continue
        for t, pt in substance(a, F).items():
            for k, pk in lever(a, F, t).items():
                p = pF * pt * pk
                if p >= PRUNE:
                    dist[code(F, t, k)] += p
    return dist
```

Running example as a tree. Probabilities multiply down each path; the winning path is bold.



Record: `code: α VF Cr`, `confidence: 0.44`, `alternatives: [α VF Su, α VF Def]`. A confidence that low says the label should be reviewed before it is trusted.

[8] Confidence matches accuracy: of all the codes given confidence 0.8, about 80% are right.

The lever and naming tables are data, not code: one file read by both the lab (Python) and the game (Rust).

1.6. Oracles

Each oracle has a rule baseline and at least one model arm. Several of the models are already in use in the reading-notes lab, a sibling project that maps passages of text to concepts.

id	question	values	rule baseline	model arm
O1	where are c, ρ, the subjects, the predicates and the markers?	spans	dependency parse + marker lexicon	GLiNER2[9] extraction
O2	$t_1 \equiv t_2$?	yes/no	lemma[10] match after coreference[11]	embedding cosine[12], cross-encoder[13]
O3	predicate type	F/V/P	modals and imperatives → P; evaluative lexicon → V; else F	GLiNER2 classification / small classifier
O4	subject quantity	s/p/u	determiners and quantifiers from the parse; proper noun → s	rarely needed
O5	does the premise predicate something of a whole proposition q_ρ, accepting or rejecting it?	A/U/none	reporting verbs with a clausal or anaphoric object, "according to", conditionals on q	GLiNER2 classification
O6	whose perspective is Z?	I/II/III	grammatical person of the source	rarely needed
O7	is $R_s = R_p$?	i/d	—	NLI
O8	which lever?	$K(F,t) \cup \{\text{none}\}$	cue words	NLI over filled-in levers

Reuse: the reading-notes lab already runs embedding models (usable for O2) and GLiNER2 (O1, O3, O5). Two things are new:

- **An NLI arm** (O7, O8): small DeBERTa-v3 NLI models (xsmall to base) run on CPU and export to ONNX,[14] so they fit that lab's latency harness.
- **A dependency parser** for the rule baselines (spaCy `en_core_web_sm` in the lab). Most of the deductive work runs on it.

[9] A small encoder that extracts spans and classifies text against labels given at run time.
[10] A word's dictionary form: "argued" → "argue".
[11] Working out which expressions refer to the same thing: "Kant … he".
[12] Turn each term into a vector and score the angle between them; near 1 means similar meaning.
[13] A model that reads both terms together and scores them as a pair; slower than embeddings, usually more accurate.
[14] A portable model format, run by ONNX Runtime in the lab and `ort` in Rust.

1.7. Gold labels and evaluation

INCEpTION currently records no PTA labels. Proposal: three features on `Argument relation`, used only on `inference` edges.

feature	values
`pta_form`	`alpha` · `beta` · `gamma` · `delta`
`pta_substance`	the substance codes for the form (`VF`, `su`, `i`, `III`, …)
`pta_lever`	the lever keyword (`CAUSE`, `GENUS`, …) or `novel`, wording in `note`

The name is derived from the three, so it isn't stored. The features map one-to-one onto pipeline stages, which allows **stage-wise evaluation**: run the pipeline with gold values substituted for everything upstream of a stage, and that stage's errors are its own. End to end, measure code top-1/top-3[15] and calibration. Statistics: each arm is compared pairwise against one baseline, with confidence intervals from resampling by passage.

A second human pass on a sample gives the ceiling: ATIP is hard for people too, and no arm should be expected to beat human agreement.

Records quote the lab's source texts, which are excerpts of published books, so they are never committed.

Decision needed: Is three extra features on `inference` edges acceptable labelling load? The alternative is a separate PTA pass over already-labelled edges, recorded under its own `Pass` (the schema's document-level record of a labelling pass).

1.8. In-game use (sketch)

The player argues with a non-player character (NPC); this pipeline types the player's arguments. The result is one input to **key extraction**: the game's step that decides which of the game's keys an argument unlocks.

What changes from the lab:

- **No gold spans.** O1 must find the claim, the reason and which is which, from the player's line *and* the dialogue: the conclusion is often the NPC's last claim, which the player supports or attacks.
- **Short, informal text.** The rule baselines will do worse than on books; the lab should keep a set of player-style notes to measure this separately.
- **Runtime.** Rust with `ort`, on a throttled CPU. Every model arm must export to ONNX, and the rule baselines must be small enough to port. The tables are already shared data (Section 1.5.7).

What key extraction gets, per argument:

- the code distribution;
- the **bound terms**: which spans filled a, b, X, Y, q, each run through the reading-notes lab's interpreters, which map a span to a concept id.

With the bound terms, an argument becomes a structured claim about concepts: "the player argued that concept Y is a CAUSE of concept X". That is the natural thing for a key to gate on, e.g. a key that needs concept X introduced *and* argued for by a lever of some type. The quality step slots in between: it reads the same record and can veto. How keys combine these is left open.

[15] Whether the gold code is the most likely code / among the three most likely.

1.9. Risks and open questions

- **Subject/predicate splits in philosophical prose.** Nominalisations and long clauses will break the parse baseline. Expected to be the largest source of O1 error.
- **Sameness is the pivot.** O2 decides form, and form decides every later candidate set. Errors there cascade; stage-wise evaluation will show how much.
- **Condensed gamma.** No good scorer for "the insertion makes the premise relevant" yet. First version: try it, report both readings, lower the confidence.
- **Explication in free text.** Players will leave the link implicit. The lever is always reconstructed anyway (O8), but a missing *premise* isn't handled. Deferred until there is player-style data.
- **Independence assumption** in the composition. Checked by calibration; if it fails, recalibrate the composed score rather than modelling dependencies.
- **A parser in Rust.** The rule baselines assume a dependency parser. The game needs one that runs under `ort` or in pure Rust; not yet surveyed.
- **Training data.** No PTA 3.1 corpus exists that I know of; Visser et al. 2021 uses the old 36-type labels and would need remapping. Fine-tuning stays deferred, as in the reading-notes lab.

1.10. Suggested order

1. Tables as data; form, substance, lever and naming as pure functions over given oracle values; composition. Testable with hand-made values.
2. Gold features in INCEpTION; stage-wise evaluation with gold oracles.
3. Rule baselines (parser, lexicons).
4. Model arms: embeddings (O2), GLiNER2 (O1, O3, O5), NLI (O7, O8).
5. Player-style data and in-game O1.

1.11. References

- Wagemans, J.H.M. (2026). *Argument Type Identification Procedure (ATIP) – Version 6.* https://www.periodic-table-of-arguments.org/argument-type-identification-procedure
- Wagemans, J.H.M. *Periodic Table of Arguments, version 3.1*, with worked examples of every type. https://www.periodic-table-of-arguments.org/pta-3-1-clickable
- Visser, J., Lawrence, J., Reed, C., Wagemans, J., Walton, D. (2021). "Annotating Argument Schemes." *Argumentation* 35, 101–139.

1.12. Appendix: the 50 PTA 3.1 types

From ATIP's Table 7. The lever column is the lever statement's pattern, with the keyword in capitals. For β, the substance is any one of those listed.

form	substance	lever	name	symbol
α	FF	Y is a SIGN of X	argument from sign	Sig
α	FF	Y is a CAUSE of X	argument from cause	Cau
α	FF	Y is an EFFECT of X	argument from effect	Ef
α	FF	Y is CORRELATED with X	argument from correlation	Cor
α	FF	Y is a MOTIVE for X	argument from motive	Mot
α	FF	Y EXCLUDES X	argument from incompatibility	Inc
α	VF	Y is a CRITERION for X	argument from criterion	Cr
α	VF	Y is the DEFINITION of X	argument from definition	Def
α	VF	Y can be SUBSUMED under X	argument from subsumption	Su
α	VV	Y is a NORM for X	argument from norm	Nor

form	substance	lever	name	symbol
α	VV	Y is an INSTANTIATION of X	argument from instantiation	Ins
α	VV	Y is a DISQUALIFIER of X	argument from disqualifier	Dis
α	PF	Y is a PRAGMATIC MOTIVATION for X	pragmatic argument	Pra
α	PF	Y is a PRIOR INVESTMENT in X	argument from waste	W
α	PF	Y is the FEASIBILITY of X	argument from possibility	Pos
α	PV	Y is a PRINCIPLED MOTIVATION for X	argument from principle	Pri
α	PV	Y is an EVALUATIVE MOTIVATION for X	argument from evaluation	Ev
α	PP	Y is a DEONTIC MOTIVATION for X	deontic argument	Deo
α	PP	Y is a TELEOLOGIC MOTIVATION for X	teleologic argument	Tel
β	ss, pp, uu	a is ANALOGOUS to b	argument from analogy	A
β	ss, pp, uu	a is SIMILAR to b	argument from similarity	Sim
β	ss, pp, uu	a is COMPARABLE to b	argument from comparison	Cpa
β	ss, pp, uu	a is EQUAL to b	argument from equality	Eq
β	ss, pp, uu	a is described METAPHORICALLY as b	metaphoric argument	Me
β	ss, pp, uu	a is COMPOSED of b	argument from composition	Cpo
β	ss, pp, uu	a is a PART of b	argument from division	Div
β	ss, pp, uu	a RANKS BELOW b	*argumentum a maiore*	Ma
β	ss, pp, uu	a RANKS ABOVE b	*argumentum a minore*	Mi
β	ss, pp, uu	a OWES RECIPROCITY to b	argument from reciprocity	R
β	ps, us, up	b is an EXAMPLE of a	argument from example	Exa
β	ps, us, up	b is a SPECIES of a	argument from species	Sp
β	sp, su, pu	a belongs to the TOTALITY of b	argument from totality	To
β	sp, su, pu	a belongs to the GENUS of b	argument from genus	G
γ	i	b is SYNONYMOUS to a and Y is SYNONYMOUS to X	argument from synonyms	Sy
γ	i	b is the OPPOSITE of a and Y is the OPPOSITE of X	argument from opposites	Op
γ	i	b is DERIVATIVE of a and Y is DERIVATIVE of X	argument from derivatives	Der
γ	i	b is INTEGRATIVE of a and Y is INTEGRATIVE of X	argument from integratives	Int
γ	i	b is the CONVERSE of a and Y is the CONVERSE of X	argument from converses	Cve
γ	d	—	non sequitur	Non

form	substance	lever	name	symbol
δ	I	Z is ETHOTICALLY AUTHORITATIVE for A	ethotic argument	Eth
δ	II	Z is CHARACTERALLY AUTHORITATIVE for U	*argumentum ad hominem*	H
δ	II	Z is a NEGATIVE MOTIVATION for A	*argumentum ad baculum*	B
δ	II	Z is an OFFERED REWARD for A	*argumentum ad carotam*	Car
δ	II	Z COMMITS the addressee to A	argument from concession	Cce
δ	II	Z is an UNDESIRABLE IMPLICATION supporting U	*argumentum ad consequentiam*	Cse
δ	III	Z is EPISTEMICALLY AUTHORITATIVE for A	argument from expert opinion	Exp
δ	III	Z is MORALLY AUTHORITATIVE for A	argument from moral authority	Mor
δ	III	Z is TESTIMONIALLY AUTHORITATIVE for A	argument from testimony	Tes
δ	III	Z is POPULARLY AUTHORITATIVE for A	*argumentum ad populum*	Pop
δ	III	Z is a PRESUMPTION in favor of A	*argumentum ad ignorantiam*	Ig